CO3-Real-World Scenario-Based Questions — Object-Oriented Engineering

1. Banking Account Management System — Classes and Inheritance

[8 Marks]

An Indian bank is developing a software system to manage different types of accounts such as SavingsAccount, CurrentAccount, and SalaryAccount. All accounts have common properties such as account number, customer name, and balance, while each account has different rules for interest calculation, minimum balance, and withdrawals.

Question:
Design an object-oriented solution using **classes and inheritance**.

1. Identify the base class and derived classes. **(2 Marks)**
2. Explain which attributes and methods should be common and which should be specialized. **(2 Marks)**
3. Write Python code demonstrating inheritance and method overriding. **(3 Marks)**
4. Explain one advantage of using inheritance in this system. **(1 Mark)**

---

2. E-Commerce Application — Composition vs Inheritance

[8 Marks]

An Indian e-commerce company is developing an order management system. An Order contains customer information, payment details, shipping information, and multiple order items. The development team is considering using inheritance between Order, Customer, Payment, and Address.

Question:
Using the concepts of **composition and inheritance**, redesign the system.

1. Identify where inheritance is appropriate. **(2 Marks)**
2. Identify where composition should be used and explain why. **(2 Marks)**
3. Design suitable Python classes. **(3 Marks)**
4. State one disadvantage of incorrectly using inheritance in this scenario. **(1 Mark)**

---

3. Logistics Platform — Mixins

[7 Marks]

An Indian logistics company has Truck, DeliveryBike, and Drone classes. Some vehicles need GPS tracking, some require notification support, and some require audit logging. The developers do not want to create a large inheritance hierarchy.

Question:
Design a solution using **Python mixins**.

1. Explain the purpose of a mixin. **(2 Marks)**
2. Design suitable GPSMixin, NotificationMixin, and AuditMixin classes. **(2 Marks)**
3. Show how they can be combined with vehicle classes. **(2 Marks)**
4. Explain one benefit and one limitation of mixins. **(1 Mark)**

---

4. Payment Gateway — PEP 484 Type Hints

[8 Marks]

A fintech application processes payments through UPI, credit cards, debit cards, and net banking. The development team wants to reduce runtime errors by introducing static type checking.

Question:
Using **PEP 484 type hints**, design the payment-processing module.

1. Add appropriate type hints for customers, amounts, and payment methods. **(2 Marks)**
2. Write a typed function for processing payments. **(2 Marks)**
3. Use Optional, List, Dict, or other appropriate typing constructs. **(2 Marks)**
4. Explain how static type checking improves reliability. **(2 Marks)**

---

5. Indian E-Commerce Application — PEP 604 Union Types

[7 Marks]

An online shopping application accepts product IDs either as integers or strings because legacy products use numeric IDs while newer products use UUID-like strings.

Question:
Refactor the application using **PEP 604 union type syntax**.

1. Define an appropriate union type using |. **(2 Marks)**
2. Write a function that accepts both types safely. **(2 Marks)**
3. Explain how PEP 604 improves readability compared with older Union[...] syntax. **(2 Marks)**
4. Give one example of incorrect type usage that static checking could detect. **(1 Mark)**

---

6. Inventory Management — Generic Types and PEP 695

[8 Marks]

A warehouse management system stores different types of items such as electronics, medicines, clothing, and grocery products. The developers want a reusable repository that can work with any product type while maintaining type safety.

Question:
Design a **generic repository** using modern Python generic syntax.

1. Explain why generics are useful in this scenario. **(2 Marks)**
2. Design a generic Repository[T]. **(2 Marks)**
3. Use **PEP 695-style generic type parameter syntax** where appropriate. **(2 Marks)**
4. Demonstrate the repository with two different product types. **(2 Marks)**

---

7. Employee Management System — Dataclasses

[7 Marks]

An Indian IT company needs to maintain employee records containing employee ID, name, department, salary, and designation. Most of the class consists of storing data rather than complex behavior.

Question:
Use Python **dataclasses** to design the employee model.

1. Explain why a dataclass is appropriate. **(2 Marks)**

2. Define the Employee dataclass with suitable type hints. **(2 Marks)**
3. Use a default value and demonstrate object creation. **(2 Marks)**
4. State two advantages of dataclasses over manually written classes. **(1 Mark)**

---

8. Healthcare Application — attrs and Validation

[8 Marks]

A hospital application maintains patient records. Each patient must have a valid patient ID, name, age, and contact number. Invalid values should be rejected when objects are created.

Question:
Design the patient model using the **attrs library**.

1. Explain why attrs is useful for this application. **(2 Marks)**
2. Define an attrs class with appropriate attributes and type hints. **(2 Marks)**
3. Add validation for at least two fields. **(2 Marks)**
4. Explain how validation improves application reliability. **(2 Marks)**

---

9. Insurance Application — Pydantic Models

[8 Marks]

An insurance company receives customer information through APIs. The input may contain incorrect data types, missing fields, or invalid values. The company wants automatic validation and conversion.

Question:
Design a **Pydantic model** for an insurance customer.

1. Define a Pydantic model with suitable fields and type hints. **(2 Marks)**
2. Add validation constraints for age, policy amount, and contact information. **(3 Marks)**
3. Explain how Pydantic differs from a basic Python class/dataclass. **(2 Marks)**
4. Give one real-world advantage of using Pydantic for API data. **(1 Mark)**

SOLID Principles

10. Food Delivery Application — Single Responsibility Principle

[7 Marks]

An Indian food delivery application has a single OrderManager class responsible for creating orders, calculating prices, processing payments, sending SMS notifications, and generating invoices. The class has become very large and difficult to maintain.

Question:
Apply the **Single Responsibility Principle (SRP)**.

1. Identify the SRP violation. **(2 Marks)**
2. Divide the class into appropriate classes. **(2 Marks)**
3. Show a suitable class design or Python implementation. **(2 Marks)**
4. Explain one maintenance benefit of your redesign. **(1 Mark)**

---

11. Payment System — Open/Closed Principle

[8 Marks]

A payment application currently supports UPI and credit cards. Whenever a new payment method such as wallets or net banking is introduced, developers modify the existing payment-processing class.

Question:
Redesign the system according to the **Open/Closed Principle (OCP)**.

1. Identify the OCP violation. **(2 Marks)**
2. Design an abstraction for payment methods. **(2 Marks)**
3. Implement at least three payment strategies without modifying the existing processor. **(3 Marks)**
4. Explain how your design supports future extensions. **(1 Mark)**

---

12. Transportation Application — Liskov Substitution Principle

[7 Marks]

A transportation company has a base class Vehicle containing a refuel() method. PetrolCar and DieselTruck implement it normally, but ElectricCar cannot use fuel and therefore raises an exception when refuel() is called.

Question:
Analyze and redesign the system according to the **Liskov Substitution Principle (LSP)**.

1. Explain why the current design violates LSP. **(2 Marks)**
2. Identify a better abstraction. **(2 Marks)**
3. Provide a class design or Python implementation. **(2 Marks)**
4. Explain how the redesign allows safe substitution. **(1 Mark)**

---

13. E-Commerce Application — Interface Segregation Principle

[7 Marks]

An e-commerce system defines a large OnlineService interface containing methods such as pay(), refund(), track_delivery(), generate_invoice(), and send_notification(). Small classes are forced to implement methods they do not need.

Question:
Apply the **Interface Segregation Principle (ISP)**.

1. Identify the design problem. **(2 Marks)**
2. Split the large interface into smaller interfaces/protocols. **(2 Marks)**
3. Show a Python implementation. **(2 Marks)**
4. Explain the advantage of your design. **(1 Mark)**

---

14. Enterprise CRM — Dependency Inversion Principle

[8 Marks]

A customer relationship management application directly creates a MySQLDatabase object inside its CustomerService class. Management now wants to support PostgreSQL and cloud databases without changing business logic.

Question:
Redesign the system using the **Dependency Inversion Principle (DIP)**.

1. Identify the DIP violation. **(2 Marks)**
2. Create an appropriate abstraction/protocol for data storage. **(2 Marks)**
3. Implement dependency injection using at least two database implementations. **(3 Marks)**
4. Explain how this improves testability. **(1 Mark)**

Design Patterns

15. E-Commerce Pricing — Strategy Pattern

[8 Marks]

An Indian e-commerce website calculates discounts differently for normal customers, premium customers, festival sales, and corporate customers. The current code contains many if-else statements.

Question:
Apply the **Strategy Design Pattern**.

1. Identify the problem with the current approach. **(2 Marks)**
2. Design a common strategy interface/protocol. **(2 Marks)**
3. Implement at least three discount strategies. **(3 Marks)**
4. Explain how a new discount strategy can be added. **(1 Mark)**

---

16. Banking Application — Factory Pattern

[8 Marks]

A banking application must create different account objects such as SavingsAccount, CurrentAccount, and FixedDepositAccount based on an account type received from the user.

Question:
Use the **Factory Design Pattern**.

1. Explain why a factory is suitable. **(2 Marks)**
2. Design the account abstraction. **(2 Marks)**
3. Implement an AccountFactory. **(3 Marks)**

4. Explain one benefit of separating object creation from business logic. **(1 Mark)**

---

17. Stock Market Application — Observer Pattern

[8 Marks]

A stock trading application monitors stock prices. When the price of a stock changes, mobile applications, email services, dashboards, and alert systems must automatically receive the updated price.

Question:
Design the system using the **Observer Design Pattern**.

1. Identify the subject and observers. **(2 Marks)**
2. Design appropriate interfaces/protocols. **(2 Marks)**
3. Implement the notification mechanism. **(3 Marks)**
4. Explain how a new observer can be added without modifying the stock class. **(1 Mark)**

---

18. Indian Payment Service — Singleton Pattern

[7 Marks]

A payment application needs one centralized configuration manager that stores API configuration, supported currencies, timeout values, and application settings. Multiple instances could lead to inconsistent configuration.

Question:
Apply the **Singleton Design Pattern**.

1. Explain why Singleton may be suitable here. **(2 Marks)**
2. Implement a Singleton configuration manager in Python. **(3 Marks)**
3. Demonstrate that multiple requests access the same instance. **(1 Mark)**
4. Mention one drawback of Singleton. **(1 Mark)**

---

19. Legacy Banking API — Adapter Pattern

[8 Marks]

A bank has an old payment gateway whose method is make_payment(amount, customer_id), while the new banking system expects pay(transaction).

Question:
Use the **Adapter Design Pattern** to integrate the legacy gateway.

1. Identify the incompatible interfaces. **(2 Marks)**
2. Define the interface expected by the new system. **(2 Marks)**
3. Implement an adapter for the legacy gateway. **(3 Marks)**
4. Explain the benefit of using an adapter instead of modifying legacy code. **(1 Mark)**

---

20. E-Commerce Checkout — Decorator Pattern

[8 Marks]

An e-commerce company wants to dynamically add features to an order such as gift wrapping, insurance, express delivery, and promotional discounts. Developers want to avoid creating separate subclasses for every possible combination.

Question:
Design the checkout system using the **Decorator Design Pattern**.

1. Explain why inheritance is not ideal for this situation. **(2 Marks)**
2. Define the base component. **(2 Marks)**
3. Implement at least three decorators. **(3 Marks)**
4. Show how decorators can be combined dynamically. **(1 Mark)**

---

21. Document Processing System — Iterator Pattern

[7 Marks]

A document management application stores thousands of documents. Users need to process documents sequentially without knowing how documents are internally stored.

Question:
Apply the **Iterator Design Pattern**.

1. Explain the purpose of the Iterator pattern. **(2 Marks)**

2. Design an iterable document collection. **(2 Marks)**
3. Implement an iterator that processes documents one at a time. **(2 Marks)**
4. Explain one advantage of hiding the collection implementation. **(1 Mark)**

---

22. Banking Transaction System — Command Pattern

[8 Marks]

An online banking application supports operations such as deposit, withdrawal, money transfer, and transaction reversal. The bank also wants to maintain transaction history and support undo functionality.

Question:
Design the system using the **Command Design Pattern**.

1. Identify the Command, Receiver, and Invoker. **(2 Marks)**
2. Design the command abstraction. **(2 Marks)**
3. Implement deposit/withdrawal commands with undo functionality. **(3 Marks)**
4. Explain how transaction history can be maintained. **(1 Mark)**

ABCs, Protocols and Domain-Driven Design

23. Banking Ledger — ABCs and Protocols (PEP 544)

[8 Marks]

An Indian banking platform supports multiple transaction types such as deposits, withdrawals, UPI transfers, and NEFT transfers. Some developers prefer inheritance-based interfaces, while others want structural typing.

Question:
Design the transaction system using **Abstract Base Classes (ABCs)** and **Protocols based on PEP 544**.

1. Explain the difference between an ABC and a Protocol. **(2 Marks)**
2. Define an abstract transaction interface. **(2 Marks)**
3. Define a Protocol for objects capable of processing transactions. **(2 Marks)**

4. Demonstrate how structural typing can be used in this application. **(2 Marks)**

---

24. Indian Banking System — Domain-Driven Design

[8 Marks]

A large Indian bank wants to develop a new banking ledger system. The application contains customers, accounts, transactions, branches, payments, and balances. The development team is mixing database code, business rules, and user-interface logic in the same classes.

Question:
Apply **Domain-Driven Design (DDD) basics** to redesign the application.

1. Identify suitable **Entities, Value Objects, and Aggregates**. **(3 Marks)**
2. Identify one or more **Repositories and Domain Services**. **(2 Marks)**
3. Explain how **Bounded Contexts** can be applied. **(2 Marks)**
4. Draw a simple domain model/architecture diagram. **(1 Mark)**

Comprehensive Enterprise-Level Scenario

25. Complete Indian Inventory & Banking Enterprise System — 8 Design Patterns

[8 Marks]

An Indian enterprise software company is developing a **domain-modelled inventory management platform** for a nationwide retail organization. The system must manage products, warehouses, suppliers, customers, orders, payments, deliveries, stock notifications, and audit logs.

The technical requirements are:

- Use **classes, inheritance, composition, and mixins** where appropriate.
- Use complete **type hints** based on PEP 484.
- Use **PEP 604 union types** where required.
- Use **generic types and modern PEP 695 syntax** where appropriate.
- Use **dataclasses, attrs, and Pydantic** appropriately.
- Follow all **SOLID principles**.

- Demonstrate the following **8 design patterns**:
    - Strategy
    - Factory
    - Observer
    - Singleton
    - Adapter
    - Decorator
    - Iterator
    - Command
- Use **ABCs and Protocols according to PEP 544**.
- Apply basic **Domain-Driven Design** concepts.
- The code should pass **mypy --strict**.
- Provide suitable **architecture and domain-model diagrams**.

Question:
Design the above system as a complete object-oriented enterprise application.

1. Identify the major domain entities, value objects, aggregates, and repositories. **(2 Marks)**
2. Explain where the **8 design patterns** should be applied. **(2 Marks)**
3. Explain how **SOLID, type hints, generics, dataclasses/attrs/Pydantic, ABCs and Protocols** will be incorporated. **(2 Marks)**
4. Draw a high-level **architecture/domain-model diagram** showing the interaction between the major components. **(1 Mark)**
5. Explain how mypy --strict improves the reliability of the application. **(1 Mark)**